

Otkrivači grešaka u distribuiranim sustavima

Bartol Gracin

Vid Joha

Matija Njegač

Distribuirani procesi

1 Asinkroni sustavi i otpornost na greške

Distribuirani sustav je otporan na greške ako nastavlja ispravno raditi i kad neki procesi zakažu. Dvije glavne paradigme u izgradnji takvih sustava su konsenzus i atomsko emitiranje. Konsenzus omogućuje skupu procesa da postignu zajedničku odluku koja ovisi o njihovim početnim vrijednostima, bez obzira na kvarove. Kod atomskog emitiranja procesi pouzdano emitiraju poruke i slažu se oko skupa isporučenih poruka i njihova redoslijeda.

Radimo u asinkronom modelu: nema gornje granice na trajanje koraka procesa ni na kašnjenje poruke (te su veličine konačne, ali neomeđene). To uglavnom uzrokuju nepredvidiva opterećenja sustava. Takav model je jednostavan i prenosiv jer ne ovisi o vremenskim pretpostavkama, ali ima važnu posljedicu: konsenzus i atomsko emitiranje ne mogu se riješiti deterministički čak ni uz jedan jedini kvar (rezultat Fischera, Lyncha i Patersona). Uzrok je upravo to što u asinkronom sustavu ne možemo razlikovati proces koji se srušio od procesa koji jako sporo odgovara.

2 Kvarovi

Promatramo skup od n procesa $Q = \{p_1, \dots, p_n\}$ povezanih pouzdanim kanalima. Proces može zakazati samo zaustavljanjem (*crash*): kažemo da se proces p srušio u trenutku t ako nakon t više ne izvodi nijednu akciju. Kvarovi su trajni; srušen proces se ne oporavlja. Proces koji se nikada ne sruši zovemo ispravnim.

Za dani tijek izvođenja Ω označavamo s $\text{Crashed}(t, \Omega)$ skup procesa koji su se srušili do trenutka t , a $\text{Up}(t, \Omega) = Q \setminus \text{Crashed}(t, \Omega)$ skup onih koji do tada rade. Pretpostavljamo da je u svakom tijeku barem jedan proces ispravan.

3 Otkrivači kvarova

Otkrivač kvarova D je distribuirani orakl koji daje naznake o kvarovima. Svaki proces p_i ima svoj lokalni modul D_i , koji prati ostale procese i održava popis onih za koje trenutno sumnja da su se srušili. Sumnja se temelji na tome koliko dugo p_i nije primio poruku od promatranog procesa.

Cijeli otkrivač je vektor $D = \langle D_{p_1}, D_{p_2}, \dots, D_{p_n} \rangle$. Pišemo $D_p(t)$ za skup procesa za koje modul procesa p sumnja u trenutku t ; ako je $q \in D_p(t)$, kažemo da p sumnja na q u trenutku t .

Bitno je da otkrivači mogu pogriješiti. Ispravan proces može završiti na popisu sumnjivih, pa kasnije biti maknut kad modul uvidi grešku. Proces se tako mogu dodavati i uklanjati s popisa proizvoljno mnogo puta, a u istom trenutku dva procesa mogu imati različite popise. Da bismo razlikovali koristan otkrivač od beskorisnog, uvodimo dva svojstva: potpunost i točnost.

4 Potpunost

Potpunost traži da otkrivač nakon nekog vremena posumnja na sve procese koji su se srušili.

- **Jaka potpunost:** nakon nekog trenutka svaki srušeni proces trajno je sumnjiv *svakom* ispravnom procesu.

$$\forall \Omega \forall p \in \text{Crashed}(\Omega) \forall q \in \text{Up}(\Omega) \exists t : \forall t' \geq t, p \in D_q(t').$$

- **Slaba potpunost:** nakon nekog trenutka svaki srušeni proces trajno je sumnjiv *nekom* ispravnom procesu.

$$\forall \Omega \forall p \in \text{Crashed}(\Omega) \exists q \in \text{Up}(\Omega) \exists t : \forall t' \geq t, p \in D_q(t').$$

Sama potpunost nije dovoljna: otkrivač koji stalno sumnja na sve procese zadovoljava jaku potpunost, a beskoristan je. Zato tražimo i točnost.

5 Točnost

Točnost ograničava greške, to jest sumnju na ispravan proces.

- **Jaka točnost:** nijedan ispravan proces nikada nije sumnjiv nijednom ispravnom procesu.

$$\forall \Omega \forall t \forall p, q \in \text{Up}(t) : p \notin D_q(t).$$

- **Slaba točnost:** neki ispravan proces nikada nije sumnjiv nijednom ispravnom procesu.

$$\forall \Omega \exists p \in \text{Up}(\Omega) \forall t \forall q \in \text{Up}(t) : p \notin D_q(t).$$

Konačna točnost

I slabu je točnost teško postići, jer otkrivač može nakratko posumnjati na ispravan proces i tek kasnije ispraviti grešku. Zato dodatno oslabljujemo zahtjev i dopuštamo rane greške, uz uvjet da one prestanu.

- **Konačna jaka točnost:** postoji trenutak nakon kojeg nijedan ispravan proces više nije sumnjiv nijednom ispravnom procesu.

$$\forall \Omega \exists t \forall t' \geq t \forall p, q \in \text{Up}(t') : p \notin D_q(t').$$

- **Konačna slaba točnost:** postoji trenutak nakon kojeg neki ispravan proces više nije sumnjiv nijednom ispravnom procesu.

$$\forall \Omega \exists t \forall t' \geq t \exists p \in \text{Up}(\Omega) \forall q \in \text{Up}(\Omega) : p \notin D_q(t').$$

6 Tipovi otkrivača

Kombiniranjem dviju vrsta potpunosti i četiri vrste točnosti dobivamo osam klasa.

Klasa	Potpunost	Točnost	Oznaka
Savršeni	jaka	jaka	$\mathcal{P}$
Konačno savršeni	jaka	konačna jaka	$\diamond \mathcal{P}$
Jaki	jaka	slaba	$\mathcal{S}$
Konačno jaki	jaka	konačna slaba	$\diamond \mathcal{S}$
Slabi	slaba	slaba	$\mathcal{W}$
Konačno slabi	slaba	konačna slaba	$\diamond \mathcal{W}$
Slaba potpunost i jaka točnost	slaba	jaka	$\mathcal{V}$
Slaba potpunost i konačna jaka točnost	slaba	konačna jaka	$\diamond \mathcal{V}$

7 Reduciranje otkrivača

Otkrivač D je reducibilan na otkrivač E ako postoji distribuirani algoritam $T_{D \rightarrow E}$ koji izlaz od D pretvara u izlaz sa svojstvima od E . Tada je svaki problem rješiv pomoću E rješiv i pomoću D , pa kažemo da je E slabiji od D i pišemo $D \succeq E$. Ako vrijedi i $E \succeq D$, otkrivači su ekvivalentni, $D \equiv E$.

Iz trivijalne redukcije (svaki proces periodički prepíše izlaz svog modula) odmah slijedi

$$\mathcal{P} \succeq \mathcal{V}, \quad \mathcal{S} \succeq \mathcal{W}, \quad \Diamond \mathcal{P} \succeq \Diamond \mathcal{V}, \quad \Diamond \mathcal{S} \succeq \Diamond \mathcal{W}.$$

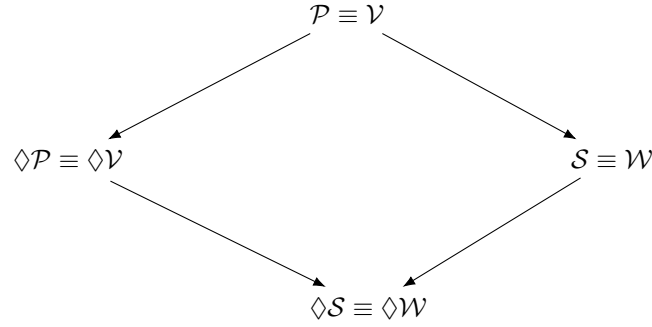
Ključna je obrnuta redukcija (Algoritam 15.1 u knjizi): svaki proces periodički šalje svoj popis sumnjivih ostalima, čime se slaba potpunost pojačava u jaku uz očuvanje točnosti. Odatle

$$\mathcal{V} \succeq \mathcal{P}, \quad \mathcal{W} \succeq \mathcal{S}, \quad \Diamond \mathcal{V} \succeq \Diamond \mathcal{P}, \quad \Diamond \mathcal{W} \succeq \Diamond \mathcal{S},$$

pa vrijede ekvivalencije

$$\mathcal{P} \equiv \mathcal{V}, \quad \mathcal{S} \equiv \mathcal{W}, \quad \Diamond \mathcal{P} \equiv \Diamond \mathcal{V}, \quad \Diamond \mathcal{S} \equiv \Diamond \mathcal{W}.$$

Posljedica je važna: dovoljno je riješiti problem za četiri klase s jakom potpunošću ($\mathcal{P}$, $\Diamond \mathcal{P}$, $\mathcal{S}$, $\Diamond \mathcal{S}$) i automatski je riješen za preostale četiri. K tome je $\mathcal{P} \succeq \mathcal{S}$ i $\Diamond \mathcal{P} \succeq \Diamond \mathcal{S}$ (Slika 1), pa se za konsenzus sve svodi na dvije netrivialne klase: $\mathcal{S}$ i $\Diamond \mathcal{S}$.



Slika 1: Hijerarhija po reducibilnosti (strelica $A \rightarrow B$ znači $A \succeq B$, tj. B je slabiji). Gore je najjači $\mathcal{P}$, dolje najslabiji $\Diamond \mathcal{S}$.

8 Konsenzus pomoću otkrivača

U konsenzusu svaki proces predlaže vrijednost, a rješenje mora zadovoljiti: konačnost (svaki ispravan proces odluči u konačnom vremenu), usuglašenost (nikoja dva ispravna procesa ne odluče različito) i valjanost (odlučena vrijednost je jedna od predloženih).

Kao što smo vidjeli, u čisto asinkronom sustavu konsenzus je nemoguć već uz jedan kvar (FLP), pa je otkrivač grešaka dodatak koji tu prepreku zaobilazi. Chandra i Toueg pokazali su kako se konsenzus rješava za svih osam klasa, a zbog ekvivalencija iz prethodnog poglavlja dovoljno je promotriti $\mathcal{S}$ i $\Diamond \mathcal{S}$. Dva rezultata određuju granicu otpornosti:

- **Uz $\mathcal{S}$ konsenzus** se rješava bez ograničenja na broj kvarova (do $n - 1$ srušenih procesa). Odgovarajući algoritam (Algoritam 15.3) smo implementirali; opisujemo ga u poglavlju 9.
- **Uz $\Diamond \mathcal{S}$ konsenzus** se rješava samo ako je većina procesa ispravna ($f < n/2$). Algoritam 15.4 koristi rotirajućeg koordinatora: u svakoj rundi jedan proces prikuplja procjene većine i predlaže novu vrijednost, a uvjet većine sprječava da se odluke različitih rundi raziđu. I ovaj smo algoritam implementirali; opisujemo ga u poglavlju 10.

Poanta je razlika u otpornosti: jaka (trajna) točnost klase $\mathcal{S}$ dovoljna je za bilo koji broj kvarova, dok konačna točnost klase $\diamond\mathcal{S}$ traži ispravnu većinu. K tome je $\diamond\mathcal{S}$ (odnosno $\diamond\mathcal{W}$) najslabiji otkrivač koji uopće rješava konsenzus, pa u praksi težimo implementirati barem $\diamond\mathcal{P}$: jači je od $\diamond\mathcal{S}$, a ostvariv je u parcijalno sinkronom sustavu (poglavlje 11).

9 Implementacija konsenzusa uz jaki detektor $\mathcal{S}$ (Algoritam 15.3)

Ovaj dio projekta (razred `ConsensusProcess`) rješava konsenzus uz jaki detektor $D \in \mathcal{S}$ i podnosi bilo koji broj kvarova (do $n - 1$ srušenih procesa). Svaki proces p drži vektor V_p svojih procjena predloženih vrijednosti (na početku sve $\perp$, osim $V_p[p] = v_p$) i vektor Δ_p vrijednosti koje je naučio u zadnjoj rundi. Upit „ $q \in D_p$ ” iz pseudokoda ostvaruje se ugrađenim detektorom (heartbeat poruke „Alive” i metoda `provjeriProces`), po istoj ideji kao Algoritam 15.7.

Algoritam 1 Konsenzus uz jaki detektor $D \in \mathcal{S}$ (procedura `PROPOSE( $v_p$ )` na procesu p)

1: $V_p \leftarrow \langle \perp, \dots, \perp \rangle$; $V_p[p] \leftarrow v_p$; $\Delta_p \leftarrow V_p$

Faza 1: za $r_p \leftarrow 1$ do $n - 1$

2: pošalji (r_p, Δ_p, p) svim procesima

3: čeka dok $\forall q$: primljeno (r_p, Δ_q, q) **ili** $q \in D_p$ ▷ upit detektoru

4: $\Delta_p \leftarrow \langle \perp, \dots, \perp \rangle$

5: **za** $k \leftarrow 1$ **do** n

6: **ako** $V_p[k] = \perp$ **i** $\exists (r_p, \Delta_q, q)$ s $\Delta_q[k] \neq \perp$

7: $V_p[k] \leftarrow \Delta_q[k]$; $\Delta_p[k] \leftarrow \Delta_q[k]$

Faza 2:

8: pošalji V_p svim procesima

9: čeka dok $\forall q$: primljeno V_q **ili** $q \in D_p$

10: **za** $k \leftarrow 1$ **do** n : **ako** $\exists$ primljeni V_q s $V_q[k] = \perp$ **tada** $V_p[k] \leftarrow \perp$

Faza 3:

11: odluči prvi element vektora V_p različit od $\perp$

9.1 Opis triju faza

Faza 1 ($n - 1$ rundi). U svakoj rundi proces emitira svoju Δ_p (vrijednosti naučene u prethodnoj rundi) i čeka poruku te runde od svakog procesa koji nije osumnjičen. Ako je za neko k vrijednost $V_p[k]$ još $\perp$, a u primljenoj poruci nije, proces je preuzima. Nakon $n - 1$ rundi svaki ispravan proces zna sve vrijednosti koje su „preživjele”.

Faza 2 (usklađivanje). Proces emitira svoj cijeli vektor V_p i čeka vektore ostalih. Ako je bilo koji primljeni vektor imao $\perp$ na mjestu k , proces i sam postavlja $V_p[k] \leftarrow \perp$. Time se izbacuju vrijednosti oko kojih se svi ne slažu; pokazuje se da barem jedno mjesto ostaje različito od $\perp$.

Faza 3 (odluka). Svi ispravni procesi odluče prvi element vektora V_p različit od $\perp$; taj je element jednak kod svih.

9.2 Implementacija u Javi

Glavna petlja (u `ConsensusTester`) vodi runde Faze 1, zatim Fazu 2 (oznaka poruke „Provjera”) i Fazu 3 (odluka). Doslovni kod:

```

1   for(int runda=1; runda < numProc; runda++){
2       System.out.println("-----");
3       System.out.println("Započeta "+runda+" runda");
4   }
```

```

5      for (int j = 0; j < numProc; j++){
6          if (j != myId){
7              process.sendMessage(j,"Consensus",runda,process.getDelta());
8              kvar();
9          }
10     }
11
12     do{
13         kvar();
14         for (int j = 0; j < numProc; j++){
15             if (j != myId){
16                 process.sendMessage(j,"Alive",runda,process.getDelta());
17             }
18         }
19         for (int j = 0; j < numProc; j++){
20             if (j != myId){
21                 process.provjeriProces(j);
22             }
23         }
24         Thread.sleep(1000);
25     }while(process.kolikoSamPorukaPrimioURundi(runda)+process.
        kolikoJeProcesaOsumnjiceno()<numProc-1);
26
27     System.out.println("-----");
28     System.out.println("Dobio sam poruke od svih ili su mi postali sumnjivi"
        );
29
30     process.ispisiSvePorukeURundi(runda);
31
32     process.obradiPoruke(runda);
33
34     process.novaRunda();
35
36 }
37 for (int j = 0; j < numProc; j++){
38     if (j != myId){
39         process.sendMessage(j,"Provjera",numProc,process.getVector());
40         kvar();
41     }
42 }
43 do{
44     kvar();
45     for (int j = 0; j < numProc; j++){
46         if (j != myId){
47             process.sendMessage(j,"Alive",numProc,process.getVector());
48         }
49     }
50     for (int j = 0; j < numProc; j++){
51         if (j != myId){
52             process.provjeriProces(j);
53         }
54     }
55     Thread.sleep(1000);
56 }while(process.kolikoSamZadnjihPorukaPrimio()+process.
        kolikoJeProcesaOsumnjiceno()<numProc-1);
57 System.out.println("Završni vektor prije provjere je "+process.V);
58 for (int j = 0; j < numProc; j++){
59     process.provjeriZadnjePoruke(j);
60 }
61 System.out.println("Završni vektor nakon provjere je "+process.V);
62 for(int i = 0; i < process.V.size() ; i++){
63     if(process.V.get(i)!=null){
64         System.out.println("Svi procesi se slažu oko vrijednosti "+process.V

```

```

65         .get(i));
66         break;
67     }
}

```

Spajanje vrijednosti u Fazi 1 (metoda obradiPoruke):

```

1  public void obradiPoruke(int runda){
2      int numProc=Delta.size();
3
4      for (int i = 0; i < numProc; i++) {
5          Delta.set(i, null);
6      }
7      System.out.println("Poruke su");
8      System.out.println(Poruke.get(runda-1));
9      for (int i = 0; i < numProc; i++) {
10         if(V.get(i)==null){
11             for(int j = 0; j < Poruke.get(runda-1).size(); j++){
12                 if(Poruke.get(runda-1).get(j)!=null){
13                     System.out.println("Idem obraditi poruku "+Poruke.get(
14                         runda-1).get(j).getMessage()+" od procesa "+Poruke.
15                         get(runda-1).get(j).getSrcId());
16                     String[] tokeni = Poruke.get(runda-1).get(j).getMessage
17                         ().trim().split("\\s+");
18                     Integer[] rezultat = new Integer[tokeni.length];
19                     rezultat[i]=tokeni[i+1].equals("null") ? null : Integer.
20                         parseInt(tokeni[i+1]);
21                     System.out.println("rezultat[i]: "+rezultat[i]);
22                     if(rezultat[i]!=null){
23                         V.set(i,rezultat[i]);
24                         Delta.set(i,rezultat[i]);
25                     }
26                 }
27             }
28         }
29         else if(V.get(i)!=null){
30             System.out.println("Vrijednost od V na indexu "+i+" nije
31                 null nego "+V.get(i));
32         }
33     }
34     System.out.println("Vektor V je ");
35     System.out.println(V);
36     System.out.println("Vektor Delta je ");
37     System.out.println(Delta);
38 }

```

Izbacivanje nepotvrđenih vrijednosti u Fazi 2 (metoda provjeriZadnjePoruke):

```

1  public void provjeriZadnjePoruke(int proces){
2      for(int i = 0; i<ZadnjePoruke.size() ; i++){
3          if(ZadnjePoruke.get(i).getSrcId()==proces){
4              System.out.println("Idem obraditi zadnju poruku "+ZadnjePoruke.
5                  get(i).getMessage()+" od procesa "+ZadnjePoruke.get(i).
6                  getSrcId());
7              String[] tokeni = ZadnjePoruke.get(i).getMessage().trim().split(
8                  "\\s+");
9              Integer[] rezultat = new Integer[tokeni.length];
10             rezultat[i]=tokeni[i+1].equals("null") ? null : Integer.parseInt
11                 (tokeni[i+1]);
12
13             if(rezultat[i]==null){
14                 V.set(i, null);
15             }
16         }
17     }
18 }

```

11
12
13
14

```

    }
  }
}

```

9.3 Dokaz ispravnosti

Otpornost. Algoritam podnosi do $n - 1$ kvarova jer nijedna faza ne traži većinu: proces čeka poruku od svakog neosumnjičenog procesa, a osumnjičene preskače.

Dogovor. Slaba točnost detektora $\mathcal{S}$ jamči da postoji ispravan proces p^* kojeg nitko nikada ne osumnjiči. Zato ga svi u svakoj rundi čekaju, pa se njegova vrijednost proširi u sve vektore i preživi Fazu 2. Svi ispravni procesi tako u Fazi 3 vide isti prvi element različit od $\perp$ i odluče jednako.

Završetak. Jaka potpunost detektora jamči da se srušeni procesi na kraju osumnjiče, pa čekanje u fazama nikada ne blokira zauvijek.

10 Implementacija konsenzusa uz konačno jaki detektor $\diamond\mathcal{S}$ (Algoritam 15.4)

U istom razredu `ConsensusProcess` implementiran je i konsenzus uz konačno jaki detektor $D \in \diamond\mathcal{S}$ (metode `runConsensus` i `phase1-phase4`). Za razliku od Algoritma 15.3, ovaj traži da je *većina procesa ispravna* ($f < n/2$). Algoritam koristi *rotirajućeg koordinatora*: u rundi r koordinatorski je proces $c = (r \bmod n) + 1$, i sve poruke idu prema koordinatoru ili od njega. Svaki proces drži procjenu odluke $estimate_p$ i vremensku oznaku ts_p (broj runde u kojoj je procjena zadnji put promijenjena).

Algoritam 2 Konsenzus uz konačno jaki detektor $D \in \diamond\mathcal{S}$ (rotirajući koordinatorski, na procesu p)

```

1:  $estimate_p \leftarrow v_p$ ;  $state_p \leftarrow \text{undecided}$ ;  $r_p \leftarrow 0$ ;  $ts_p \leftarrow 0$ 
   cobegin
   Zadatak 1: dok  $state_p = \text{undecided}$ 
2:    $r_p \leftarrow r_p + 1$ ;  $c_p \leftarrow (r_p \bmod n) + 1$  ▷ koordinatorski runde
   Faza 1:  $p$  šalje  $(p, r_p, estimate_p, ts_p)$  koordinatorski  $c_p$ 
   Faza 2: ako  $p = c_p$ 
3:   čekaj dok ne primiš procjene od  $\lceil (n+1)/2 \rceil$  procesa
4:    $t \leftarrow$  najveći  $ts_q$  među primljenima
5:    $estimate_p \leftarrow$  neki  $estimate_q$  s tim  $t$ ; pošalji  $(p, r_p, estimate_p)$  svima
   Faza 3: čekaj dok ne primiš  $(c_p, r_p, estimate_{c_p})$  ili  $c_p \in D_p$ 
6:   ako primljeno:  $estimate_p \leftarrow estimate_{c_p}$ ;  $ts_p \leftarrow r_p$ ; pošalji  $ack$ 
7:   inače pošalji  $nack$  ▷ sumnja na koordinatorski
   Faza 4: ako  $p = c_p$ 
8:   čekaj  $\lceil (n+1)/2 \rceil$  odgovora; ako su svi  $ack$ : R-broadcast  $(p, r_p, estimate_p, decide)$ 
   Zadatak 2: kad primiš  $(q, r_q, estimate_q, decide)$ 
9:   ako  $state_p = \text{undecided}$ : odluči  $estimate_q$ ;  $state_p \leftarrow \text{decided}$ 
   coend

```

10.1 Opis četiriju faza

Faza 1. Svaki proces šalje koordinatorski svoju trenutnu procjenu i vremensku oznaku $(estimate_p, ts_p)$.

Faza 2. Koordinator čeka procjene od većine procesa, odabere onu s najvećom vremenskom oznakom (najsvežiju) i emitira je svima kao novu procjenu.

Faza 3. Svaki proces čeka novu procjenu koordinatora. Ako je primi, prihvati je i pošalje *ack*; ako posumnja na koordinatora (upit detektoru D_p), pošalje *nack*.

Faza 4. Koordinator čeka odgovore većine. Ako je većina poslala *ack*, vrijednost je „zaključana” i koordinator emitira zahtjev za odluku (*decide*). Svaki proces koji primi *decide* odlučuje tu vrijednost.

Uvjet većine ($f < n/2$) je nužan: dvije faze čekaju odgovore od $\lceil (n+1)/2 \rceil$ procesa, pa se presjek bilo koje dvije većine preklapa u barem jednom procesu, čime se sprječava da odluke različitih rundi budu proturječne. (U kodu su procesi indeksirani od 0, pa je koordinator $r \bmod n$.)

10.2 Implementacija u Javi

Glavna petlja rotira koordinatora dok se ne odluči (metoda `runConsensus`):

```
1 public void runConsensus(){
2
3     System.out.println("P" + myId + " estimate = " + estimate);
4     while(state == 0){
5         receivedPhase1Messages.clear();
6         receivedPhase2Messages.clear();
7         round++;
8         int coordinatorId = (round%n);
9
10
11         System.out.println("Round " + round + " coordinator = " +
12             coordinatorId);
13
14         phase1_sendEstimate(coordinatorId);
15         phase2_coordinatorSelect(coordinatorId);
16         phase3_ackNack(coordinatorId);
17         phase4_decide(coordinatorId);
18
19         try {
20             Thread.sleep(1000); // mali delay da vidiš roundove
21         } catch (InterruptedException e) {}
22
23     }
24     System.out.println("FINAL DECISION = " + estimate);
25 }
```

Faza 1 i Faza 2 (koordinator skuplja većinu i bira procjenu s najvećom oznakom):

```
1 public void phase1_sendEstimate(int coordinatorId) {
2
3     // pakiramo estimate + timestamp (estimateRound)
4     String payload = estimate + " " + estimateRound;
5     if (coordinatorId != myId)
6         linker.sendMsg(coordinatorId, "PHASE1", payload);
7
8     System.out.println("P" + myId +
9         " -> PHASE1 sent to " + coordinatorId +
10         " value=" + estimate +
11         " ts=" + estimateRound);
12 }
13
14 public void phase2_coordinatorSelect(int coordinatorId) {
15     // samo koordinator radi Phase 2
16 }
```



```

16         if (myId != coordinatorId) return;
17
18         System.out.println("P" + myId + " je coordinator u rundi " + round);
19
20         int bestEstimate = estimate;
21         int bestRound = estimateRound;
22         int majority = (n/2) +1;
23         while(receivedPhase1Messages.size() < majority-1)
24             {
25                 try {
26                     Thread.sleep(100);
27                 } catch (InterruptedException e) {
28                     Thread.currentThread().interrupt();
29                     break;
30                 }
31             }
32         List<Msg> snapshot = new ArrayList<>(receivedPhase1Messages);
33         for (Msg m : snapshot) {
34
35             String[] parts = m.getMessage().trim().split("\\s+");
36
37             int value = Integer.parseInt(parts[0]);
38             int ts = Integer.parseInt(parts[1]);
39
40             if (ts > bestRound) {
41                 bestRound = ts;
42                 bestEstimate = value;
43             }
44         }
45
46         // update coordinator state
47         estimate = bestEstimate;
48         estimateRound = round;
49
50         System.out.println("Coordinator picked: " + estimate);
51
52         // šalje svima novu vrijednost
53         String payload = estimate + " " + estimateRound;
54
55         linker.broadcastToAll("PHASE2", payload);
56
57         Msg self = new Msg(myId, myId, "PHASE2",payload);
58
59         receivedPhase2Messages.add(self);
60     }

```

Faza 3 (prihvat procjene i slanje ack/nack) i Faza 4 (odluka koordinatora na temelju većine ack-ova):

```

1     public void phase3_ackNack(int coordinatorId) {
2
3         Msg coordinatorMsg = null;
4
5         while(receivedPhase2Messages.isEmpty())
6             {
7                 try {
8                     Thread.sleep(100);
9                 }
10                catch(InterruptedException e)
11                {
12                    Thread.currentThread().interrupt();
13                }
14            }
15        // tražimo PHASE2 poruku od koordinatora

```

```

16 List<Msg> snapshot = new ArrayList<>(receivedPhase2Messages);
17 for (Msg m : snapshot) {
18
19     if (m.getSrcId() == coordinatorId && m.getDestId() == myId) {
20
21         coordinatorMsg = m;
22
23         String[] parts = m.getMessage().trim().split("\\s+");
24
25         int value = Integer.parseInt(parts[0]);
26         int valueRound = Integer.parseInt(parts[1]);
27
28         // prihvati vrijednost
29         estimate = value;
30         estimateRound = round;
31
32         System.out.println("P" + myId +
33             " ACCEPT PHASE2 from " + coordinatorId +
34             " value=" + estimate);
35
36         break;
37     }
38 }
39
40 // ako smo dobili poruku -> ACK
41 if (coordinatorMsg != null || coordinatorId == myId) {
42     if (coordinatorId != myId)
43         linker.sendMsg(coordinatorId, "ACK", round + "");
44     else
45         receivedPhase3Replies.add(new Msg(myId, myId, "ACK", round + ""));
46
47     System.out.println("P" + myId + " -> ACK to " + coordinatorId);
48 }
49 // ako nismo dobili -> NACK
50 else {
51     if (coordinatorId != myId)
52         linker.sendMsg(coordinatorId, "NACK", round + "");
53
54     System.out.println("P" + myId + " -> NACK to " + coordinatorId);
55 }
56 }
57
58 public void phase4_decide(int coordinatorId) {
59
60     // samo koordinator odlučuje
61     if (myId != coordinatorId) return;
62
63     int ackCount = 0;
64     int nackCount = 0;
65
66     int mmajority = (n/2)+1;
67     while(receivedPhase3Replies.size() < mmajority)
68     {
69         try { Thread.sleep(100); }
70         catch(InterruptedException e) { Thread.currentThread().interrupt(); }
71     }
72     List<Msg> snapshot = new ArrayList<>(receivedPhase3Replies);
73     for (Msg m : snapshot) {
74         if (m.getSrcId() != coordinatorId) {
75             if (m.getTag().equals("ACK")) ackCount++;
76             else if (m.getTag().equals("NACK")) nackCount++;

```

```

77         }
78     }
79
80     System.out.println("P" + myId + " ACK=" + ackCount + " NACK=" +
        nackCount);
81
82     // majority rule
83     int majority = (n / 2) + 1;
84     if (ackCount >= majority) {
85         System.out.println("P" + myId + " DECIDES value = " + estimate);
86         linker.broadcastToAll("DECIDE", estimate + " " + round);
87         state = 1; // decided
88     }
89     receivedPhase3Replies.clear();
90 }

```

Odluka se prima u `handleMsg` (Zadatak 2 iz pseudokoda):

```

1         else if (tag.equals("DECIDE")) {
2             receivedDecideMessages.add(m);
3             String[] parts = m.getMessage().trim().split("\\s+");
4             estimate = Integer.parseInt(parts[0]);
5             state = 1;
6             System.out.println("P" + myId +
7                 " received DECIDE from " + src + " value=" + estimate);
8         }

```

10.3 Dokaz ispravnosti

Potreba za većinom. Faze 2 i 4 čekaju odgovore od $\lceil (n+1)/2 \rceil$ procesa. Dvije se većine uvijek preklapaju u barem jednom procesu, pa se jednom „zaključana” vrijednost prenosi u sve kasnije runde i nikoja dva procesa ne mogu odlučiti različito (usuglašenost). Ako većina nije ispravna, čekanje bi moglo blokirati zauvijek, pa algoritam zahtijeva $f < n/2$.

Završetak. Konačna slaba točnost detektora $\diamond S$ jamči da nakon nekog vremena postoji ispravan koordinator kojeg nitko ne sumnjiči. Kad on dođe na red, njegovu procjenu svi prihvate (*ack*), vrijednost se zaključa i emitira *decide*, pa svi ispravni procesi odluče.

11 Implementacija otkrivača $\diamond P$ (Algoritam 15.7)

Savršen otkrivač $\mathcal{P}$ ne možemo implementirati u parcijalnoj sinkroniji: dok ne znamo gornju granicu kašnjenja, svaki fiksni istek vremena može biti prekratak i osumnjičiti spor, ali ispravan proces. Najbolje što postizemo jest da takve greške *na kraju* prestanu, a to je upravo $\diamond P$.

Parcijalna sinkronost znači da je sustav u početku asinkron, ali nakon nekog nepoznatog trenutka postane sinkron, s omeđenim kašnjenjima. Svaki proces p drži zadani istek vremena za svaki drugi proces i mjeri vrijeme lokalnim satom koji broji korake (otkucaje). Koriste se varijable: $Output_p$ (popis sumnjivih, na početku prazan) i $\Delta_p(q)$ (trajanje isteka vremena procesa p za q).

11.1 Opis triju zadataka

Zadatak 1 (otkucaji srca). Svaki proces svima periodički šalje poruku „živ sam”. Kad se proces sruši, prestane ih slati.

Zadatak 2 (istek vremena). Ako p od q nije čuo ništa u zadnjih $\Delta_p(q)$ otkucaja, doda q među sumnjive.

Algoritam 3 Timeout implementacija $D \in \Diamond\mathcal{P}$ u modelu parcijalne sinkronosti (svaki proces p)

```

1:  $Output_p \leftarrow \emptyset$ 
2: for all  $q \in Q$  do  $\Delta_p(q) \leftarrow$  zadani istek vremena
3: end for
   cobegin
     Zadatak 1: periodički
4:   pošalji „ $p$  je živ” svima
     Zadatak 2: periodički, za svaki  $q \in Q$ 
5:   ako  $q \notin Output_p$  i  $p$  nije primio „ $q$  je živ” u zadnjih  $\Delta_p(q)$  otkucaja
6:      $Output_p \leftarrow Output_p \cup \{q\}$ 
     Zadatak 3: kad primiš „ $q$  je živ”
7:   ako  $q \in Output_p$ 
8:      $Output_p \leftarrow Output_p \setminus \{q\}$  ▷  $p$  se kaje
9:      $\Delta_p(q) \leftarrow \Delta_p(q) + 1$  ▷ i poveća istek vremena
   coend

```

Zadatak 3 (kajanje). Ako p primi poruku od procesa kojeg sumnjiči, zna da je istek vremena bio preuranjen: makne q iz sumnjivih i *poveća* $\Delta_p(q)$, da idući put bude strpljiviji.

11.2 Implementacija u Javi

Otkrivač smo izdvojili u zaseban razred `FailureDetector` unutar okvira za razmjenu poruka. Stanje odgovara varijablama iz pseudokoda: `Output` je skup sumnjivih, `delta[q]` istek vremena za q , `lastAlive[q]` otkucaj zadnje poruke od q , a `clock` lokalni sat.

```

1  // Zadatak 1: posalji "ziv sam" svima
2  public void sendHeartbeats() {
3      if (crashed) return;
4      broadcastMsg("alive", (int) clock);
5  }
6
7  // Zadatak 2: istek vremena -> sumnja
8  public synchronized void checkTimeouts() {
9      for (int q = 0; q < N; q++) {
10         if (q == myId) continue;
11         if (!Output.contains(q) && (clock - lastAlive[q]) > delta[q]) {
12             Output.add(q);
13         }
14     }
15 }
16
17 // Zadatak 3: kad primim "ziv sam" od q -> kajanje + veci timeout
18 public synchronized void handleMsg(Msg m, int q, String tag) {
19     if (tag.equals("alive")) {
20         lastAlive[q] = clock;
21         if (Output.contains(q)) {
22             Output.remove(q);
23             delta[q] = delta[q] + 1;
24         }
25     }
26 }

```

Zadaci 1 i 2 izvode se u glavnoj petlji, gdje jedan prolaz odgovara jednom otkucaju (`clock++`). Zadatak 3 izvodi se asinkrono, u dretvama koje slušaju poruke, pa su metode koje dijele stanje označene s `synchronized`.

11.3 Dokaz ispravnosti

Jaka potpunost. Srušeni proces trajno prestaje slati otkucaje. Razlika `clock - lastAlive[q]` tada raste bez granice i prije ili kasnije premaši $\Delta_p(q)$, pa q ulazi u $Output_p$. Iz $Output_p$ se izlazi samo primitkom nove poruke od q , koje više nema, pa sumnja ostaje trajna.

Konačna jaka točnost. Nakon svake pogrešne sumnje na ispravan proces q , pri sljedećoj poruci od q istek $\Delta_p(q)$ poraste za jedan. Kad sustav postane sinkron, kašnjenja imaju konačnu gornju granicu; $\Delta_p(q)$ nakon konačno mnogo grešaka premaši tu granicu i pogrešnih sumnji više nema.

11.4 Testiranje

Otkrivač smo testirali s tri procesa i scenarijima koji izdvajaju svako svojstvo (Tablica 1).

Scenarij	Opažanje
Kvar procesa p_2 u otkucaju 10	p_0 i p_1 počnu sumnjati na p_2 oko otkucaja 15 i zadrže ga u $Output$ do kraja; ispravne procese nikada ne sumnjiče. Potvrđuje jaku potpunost.
Spor proces p_2 (otkucaj srca svakih 8 otkucaja)	p_0 i p_1 ga isprva više puta pogrešno osumnjiče i pokaju se, a Δ raste $5 \rightarrow 6 \rightarrow 7 \rightarrow 8$; nakon toga sumnje prestaju. Potvrđuje konačnu jaku točnost.

Tablica 1: Rezultati testiranja otkrivača $\diamond\mathcal{P}$.

Drugi scenarij dobro pokazuje zašto je ovo $\diamond\mathcal{P}$, a ne $\mathcal{P}$. Otkrivač pogriješi dok ne nauči pravi ritam sporog procesa, ali čim Δ dostigne stvarni razmak otkucaja, greške staju. Savršen otkrivač tu ne bi pogriješio nijednom, ali u parcijalnoj sinkroniji nije ostvariv.

Literatura

- [1] Kshemkalyani A.D., Singal M. Distributed Computing: Principles, Algorithms and Systems. Cambridge University Press, 2011.